

Redis from scratch: keys, data types, expiry, and durability

Meet Redis as a running data-structure server, read its basic commands, and distinguish a disposable cache from data whose loss changes the product.

🕒 45 minutes 📖 Read, try, and explain 💻 Saved on your computer

WHAT YOU'LL UNDERSTAND

- ✓ Trace a command from application to Redis and back.
- ✓ Choose strings, hashes, lists, sets, and sorted sets from concrete operations.
- ✓ Explain key expiration and memory eviction.
- ✓ Distinguish cache correctness from persistence and atomic command behavior.

A few words we'll use

No need to memorize these. Come back whenever you need them.

Redis

A data-structure server that applications access using commands.

Client library

Application code that manages communication with another service and exposes convenient operations.

TTL

Time to live: an expiration duration associated with stored data.

Eviction

Removal of data under a memory-management policy.

Persistence

Saving data outside temporary process memory so it can be recovered under a configured policy.

Atomic command

A command whose operation is indivisible with respect to competing operations under the server's execution rules.

01 Redis is a program your application talks to

Pocket Notes already keeps accepted notes in a database. Imagine its public home page repeatedly asks for the same small summary. We can keep a copy in Redis and avoid doing the same database work for every visit. First understand what Redis is: a server process that receives commands, manages data, and sends replies.

The notes application uses a Redis client library to connect and issue those commands. Redis is not a keyword added to a SQL query, and installing a library does not by itself start a Redis server. In a local setup both programs can run on one machine; in a larger service they may be separate network participants.

Think of a staffed desk holding labeled working cards. You ask the desk for a card or request a change. The analogy stops at guarantees: Redis has specific data types, concurrency, memory, and persistence behavior. A desk metaphor cannot prove that a saved value survives a crash.

02 Read a small command conversation

These commands illustrate the Redis command interface. They are examples to trace, not evidence that a Redis instance has been installed or tested on your machine.

```
SET demo:note:41 "Garden"  
GET demo:note:41  
DEL demo:note:41  
GET demo:note:41
```

Read the commands as a conversation:

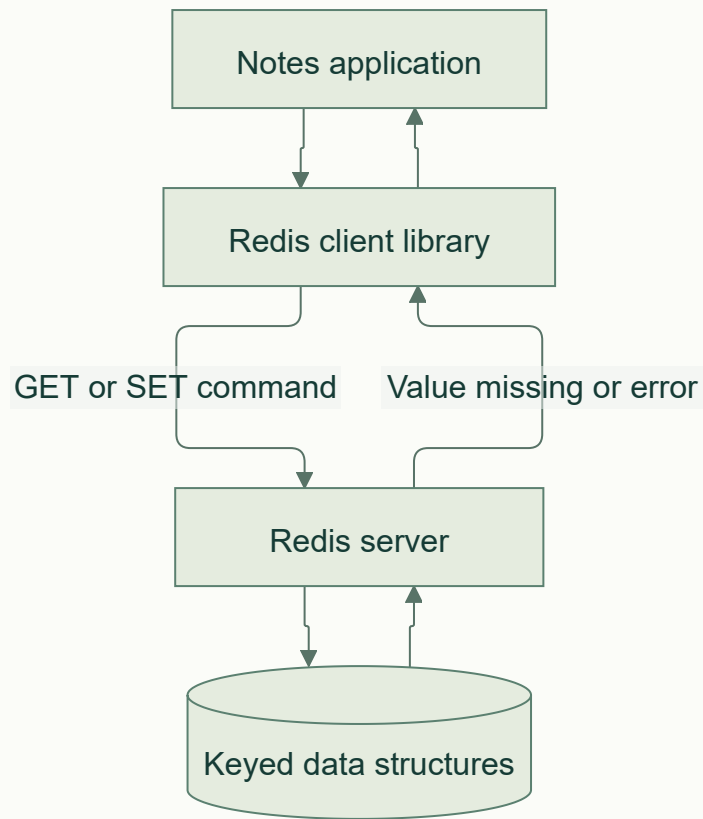
Command	Effect or reply in this successful sequence
SET demo:note:41 "Garden"	Store the string Garden under that key
GET demo:note:41	Return Garden
DEL demo:note:41	Remove the existing key
GET demo:note:41	Report no value, commonly shown as nil

Missing is different from an existing empty string, an error, or a timeout. Keep those outcomes separate in the application.

The colons are a naming convention, not automatic nested folders. A name such as demo:note:41 makes the scope understandable. In a shared application, include every identity needed to avoid mixing different accounts or representations. Keep command names separate from untrusted values through the client library's argument interface.

SYSTEM VIEW

The application sends commands to a separate server role



The client library and Redis server are different roles even when they run on the same laptop.

1. Create a connection through the client library.
2. Send a command and separate arguments.
3. Let Redis apply its data-type rules.
4. Handle success, missing data, timeout, and error distinctly.

Diagram labels

- GET or SET command
- Value missing or error
- Notes application
- Redis client library
- Redis server
- Keyed data structures

03 Choose the structure from the operation

A string holds a sequence of bytes; it can contain text, encoded JSON, or a supported integer representation. A hash stores named fields under one key. A list keeps values in sequence. A set keeps unique members without promising application-visible ordering. A sorted set associates unique members with numeric scores for ordered retrieval.

Need	Candidate structure	Example operation
One prepared public summary	String	GET the known key
A small group of deployment fields	Hash	HGET a named field
Ordered recent values	List	Read a bounded slice
Unique labels attached to a note	Set	Add or test membership
Ranked items with scores	Sorted set	Retrieve a score range

Redis also has other structures, including streams, but the first five already illustrate the principle. Choose by required operations and measured size. Storing a huge JSON string and replacing it for every tiny change has different costs from changing one hash field.

04 Worked example: update a field and a counter

Consider these illustrative commands:

```
HSET demo:deployment:301 status ready region eu
HGET demo:deployment:301 status
SET demo:views 0
INCR demo:views
```

HSET records two fields in one hash. HGET returns ready for the status field. INCR adds one to the integer value, so the counter becomes 1. Redis checks types: using an unsuitable command on a key with an incompatible type produces an error rather than treating every representation as interchangeable.

Why use INCR rather than GET, add in application code, then SET? Two callers can both read 0 and each write 1, losing one increment. A supported atomic increment avoids that split operation. But sending the same increment twice still increments twice. Atomic does not mean duplicate-safe: a retried view event needs its own product policy if duplicate counting matters.

05 Expiration removes a key; it does not refresh it

SET demo:summary "Garden" EX 60 stores a string with a sixty-second expiration. TTL demo:summary asks about remaining lifetime in seconds:

Example TTL result	Meaning
35	About 35 seconds of remaining lifetime
-1	Key exists without an expiration
-2	Key does not exist

The negative values are special results, not an ordinary negative countdown. Redis does not regenerate a value when its lifetime ends.

After the expiry time, the key is no longer usable as a live cached answer. The next cache miss may trigger a fresh database read. Redis does not know how to regenerate this summary simply because EX was supplied. That work belongs to the application.

Also, sixty seconds since cache insertion is not necessarily sixty seconds since the source changed. A slow fill could insert an already-old value. Replacing a key with plain SET normally discards its earlier TTL unless an appropriate option preserves it. Always review expiration when changing write commands; accidentally immortal cache entries are a common source of stale results.

WORK THROUGH AN EXAMPLE

Changing a value can change its expiration too

This paper command trace uses idealized elapsed times. Real TTL display depends on when the command is observed; no live Redis execution is claimed.

STEP 1 OF 5

Store an expiring summary

Time	0 seconds
Command	SET demo:summary Garden EX 60
TTL	About 60 seconds

The initial SET supplies both a value and an explicit expiration.

STEP 2 OF 5

Read its remaining lifetime

Time	25 seconds
Command	TTL demo:summary
Reply	About 35 seconds

After 25 seconds, the key can still be a live cached value in this simple trace.

STEP 3 OF 5

Replace it without expiry

Time	25 seconds
Command	SET demo:summary SpringGarden
TTL reply	-1 – no expiration

Plain SET normally discards the previous expiration. A correct new value can accidentally become an unbounded-lived cache entry.

STEP 4 OF 5

Write the intended policy explicitly

Time	25 seconds
Command	SET demo:summary SpringGarden EX 60
Expected expiry	Around time 85 seconds

If the desired rule is 60 seconds from this replacement, include EX 60 in that write. This begins a new lifetime.

STEP 5 OF 5

After that lifetime

Time	After 85 seconds
TTL reply	-2 – key absent
Next work	Application miss handling

Once expired, the key is absent. The application must fetch or rebuild the summary if needed.

What changed?

Review the lifetime every time you change the write command. Expiration removes a value; it does not refresh the source or rebuild the result.

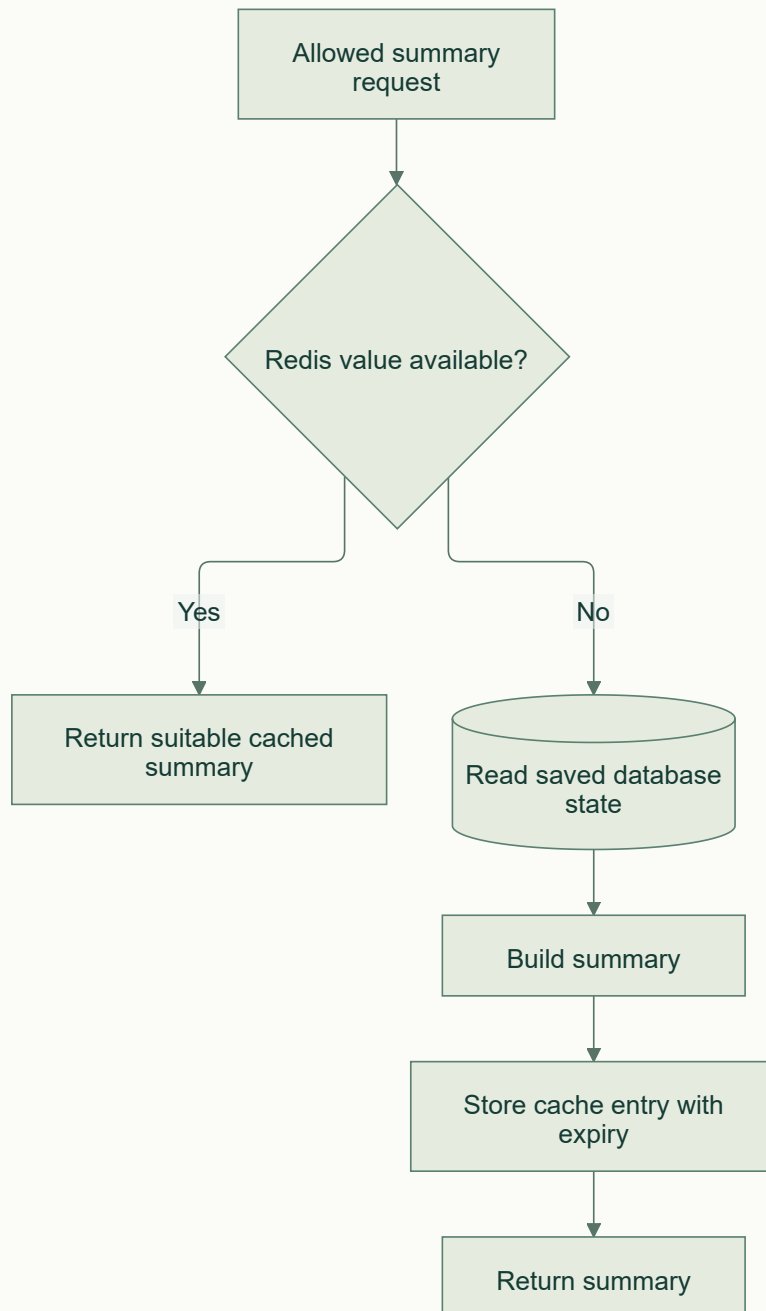
06 Follow a miss, a fill, and a later hit

Use cache-aside for the public summary. First the application checks Redis. If the key exists and its representation is valid for this request, return it. If the key is absent, read the saved database records, compute the summary, store it with a chosen expiration, and return it. A later request can reuse that stored result.

The database remains the source of truth here: the place whose accepted state decides the real saved notes. Losing the cache should lose a shortcut, not the only copy of a note. Cache failure can fall back to the database within a bounded load policy; sending an unlimited burst to storage may overload it.

An edit changes the saved note first under its normal rules. Then the application can remove or refresh affected cached representations. There is still a race if an older in-flight read fills the cache afterward. Later freshness lessons examine versions and coordination; Redis's presence alone does not solve that ordering problem.

A missing cache entry leads back to saved data



This example caches a derived public representation. An unavailable Redis server additionally needs a bounded fallback policy.

1. Attempt the known scoped key.
2. Treat missing cache data as a miss.
3. Rebuild from saved records.
4. Store an expiring reusable result and return it.

Diagram labels

- Yes
- No
- Allowed summary request
- Redis value available?
- Return suitable cached summary
- Read saved database state
- Build summary
- Store cache entry with expiry
- Return summary

07 A fast working copy still has limits

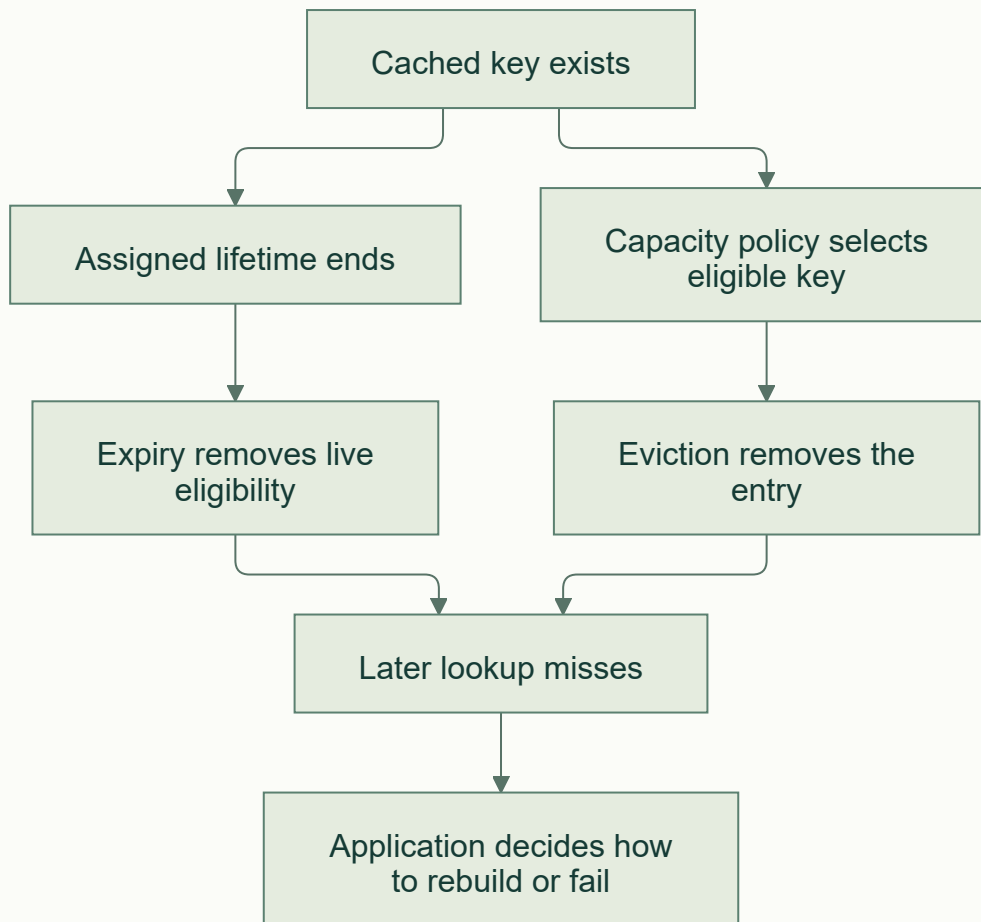
Redis commonly keeps its working dataset in memory, avoiding disk access for many ordinary operations. This is a reason to investigate it for low-latency work, not a guarantee that every command is cheap. Large values, large scans, network delay, and busy processing all affect response time.

Memory has a budget. An eviction policy may remove eligible keys when the configured limit is reached; a no-eviction policy can instead reject writes that need more memory. Eviction differs from expiry: expiry follows a key's time condition, while eviction responds to capacity policy. Applications using Redis as a cache must expect misses from either cause.

Decide which keys are replaceable. Mixing disposable summaries and irreplaceable work records under an unsuitable eviction policy can lose important state. Bounded result sizes and explicit memory observations matter more than assuming that an in-memory product has infinite capacity.

SYSTEM VIEW

The same miss can have different causes



A no-eviction policy may reject writes instead of removing keys. These cache-removal paths do not state that a durable source record was deleted.

1. Separate time-based expiry from capacity-driven eviction.
2. Expect either policy to affect whether a cached key is usable.
3. Handle a miss or a write error according to the store's actual configuration.

Diagram labels

- Cached key exists
- Assigned lifetime ends
- Capacity policy selects eligible key
- Expiry removes live eligibility
- Eviction removes the entry
- Later lookup misses
- Application decides how to rebuild or fail

08 A cache and a primary store need different recovery promises

Redis supports persistence mechanisms such as snapshots, which periodically record dataset state, and an append-only file, which records changes for recovery. The exact policy determines which acknowledged changes might be absent after a failure and how long restoration takes. Turning on persistence is not a universal zero-loss promise.

If Redis only caches public summaries, rebuilding them may be acceptable. If it is the only store for user sessions or job state, a loss changes user or workflow behavior. A session loss might require signing in again; lost payment or job state may be far less acceptable. State the loss tolerance for the actual data.

Replicas and backups address different failure stories. Another live copy can help with some failures, while retained recoverable history may help after an accidental change. Configure, monitor, and test the required recovery path. Do not infer safe failover or complete backup coverage from the name Redis.

09 Atomic operations do not cover an entire application

A single supported command can protect its own operation, but a sequence of separate application commands can have gaps. Redis transactions group queued commands for execution without another client's commands interleaving. They do not provide SQL-style rollback of every successful command when a later command encounters a runtime error.

Nor does a Redis operation atomically commit an unrelated SQL update or external API call. If the database accepts a note but cache invalidation fails, the design needs a freshness or recovery policy. If a response disappears after INCR succeeds, retrying may increment again. Separate the question "could another operation interleave?" from "could this whole request run twice?"

For this chapter, keep the correctness rule small: cache entries are derived, scoped, and replaceable; important saved records remain in the database. Learn a more complex Redis role only when its guarantees are understood and needed.

Interview phrasing: "I am using Redis for a derived public summary, so losing that entry can cause a miss rather than losing an accepted note. I still need bounded fallback load, correct key scope, and an explicit freshness policy."

10 Worked example: caching an AI deployment configuration

An AI gateway repeatedly reads deployment 301's ready configuration: model version 12, allowed limits, and region. Suppose this complete configuration is revision 4. A cached copy can avoid repeated registry reads. Use a key that includes the account and configuration revision, such as `config:team8:deployment301:cfg4`. The model version and configuration revision are different identities: changing a limit or region creates a new configuration revision even if the model stays at version 12. Check the caller's access; knowing the key is not authorization.

Step one: resolve the allowed deployment and current configuration version. Step two: check its cache key. Step three: on a miss, load the permitted registry data and cache that representation. Step four: when configuration changes, use the new version and retire old copies under policy. Versioned keys do not eliminate the need to learn which version is current.

Do not casually cache all model answers under prompt text alone. Answers may depend on user permissions, model version, retrieved private documents, settings, and time. Similar text does not establish safe interchangeability. Start with a clearly reusable configuration record; define all inputs and privacy boundaries before considering generated-answer caching.

YOUR TURN · 12 MINUTES

Try it yourself.

Trace SET demo:views 0, then two INCR commands, then GET. Explain the expected final value, why replacing each increment with a separate GET/SET can lose an update, and why retrying an uncertain INCR can overcount. Then decide whether this counter may be lost in your product.

Your explanation or sketch

Use your own words. A few sentences, a list of steps, or a simple sketch description is enough.

What to include ▾

Show an example answer →

A SMALL CHECK, NOT A TEST TO FEAR

Does the idea make sense?

Try these questions. The explanation matters more than getting it right the first time.

1

Redis client library and Redis server are the same thing:

☐ True

☐ False

2

What does a TTL automatically do for our summary?

☐ Recompute it from SQL

☐ Expire the key according to its time condition

☐ Prevent all stale reads

3

Two successful INCR commands on an initial zero produce:

☐ 0

☐ 1

☐ 2

4

Why is Redis with persistence not automatically a zero-loss store?

Explain it as if you were helping a friend understand.

5

What belongs in an AI cache-key decision besides prompt text?

Explain it as if you were helping a friend understand.



THE ONE IDEA TO KEEP

Know the key, the operation, and the role of the value. Then ask what expiration, repetition, and loss mean.

OPTIONAL · WHEN YOU WANT ANOTHER EXPLANATION

Go a little further.

You can finish this lesson without opening these links. Videos and articles need an internet connection.



ByteByteGo · Optional online resource

What is a Load Balancer? [↗](#)

Locate the traffic distributor in a small multi-server app before exploring its many types.

Free public guide; linked courses or other destinations may have separate access requirements. · Link checked



ByteByteGo · Optional online resource

Proxy vs Reverse Proxy [↗](#)

Compare which side of a conversation the intermediary serves, after the main load-balancing walkthrough.

Free public guide; linked courses or other destinations may have separate access requirements. · Link checked



ByteByteGo · Optional online resource

Top 5 Caching Strategies [↗](#)

Optional visual vocabulary for read/write cache paths. Our original guide supplies freshness and failure reasoning.

Free public guide; linked courses or other destinations may have separate access requirements. · Link checked

